# Schema Evolution Workflow on S3-Compatible Object Storage

## 1. Introduction

Modern data systems frequently evolve over time. In practice, datasets rarely remain fixed, and new fields are often introduced as applications grow. A common challenge is therefore maintaining compatibility between older and newer dataset versions while still allowing unified analytics queries.

The objective of this project was to implement and benchmark a simple schema evolution workflow on S3-compatible object storage. The implementation combines versioned Parquet datasets, schema contracts, MinIO object storage, and backward-compatible reads.

The project simulates a small cloud-style data lake architecture where multiple dataset versions remain queryable through a unified schema. Two dataset versions were generated:

- v1, containing the baseline schema;
- v2, introducing an additional nullable column.

The workflow was benchmarked across multiple dataset sizes in order to evaluate:

- mixed-version reads;
- query runtime;
- storage behavior;
- transfer performance.

The implementation was developed in Python using PyArrow, Pandas, Boto3, Docker, and MinIO.

## 2. System Architecture

The project uses MinIO as a local S3-compatible storage system. The datasets are stored as Parquet files inside a bucket named ccbd-data.

The storage structure is organized as follows:

```
ccbd-data/
├── curated/
│   └── events/
│       ├── v1/
│       └── v2/
└── published/
    └── events/
        └── schema.json
```

The curated/ prefix contains the Parquet datasets, while the published/ prefix contains the schema contract used by the reader during validation.

The workflow is composed of five main scripts:

- dataset_gen.py generates synthetic datasets;
- upload.py uploads datasets to MinIO;
- download.py downloads objects locally;
- reader_s3.py reads and validates datasets from object storage;
- bench.py executes the complete benchmark workflow.

The benchmark pipeline automatically:

1. generates datasets;
2. uploads datasets to MinIO;
3. downloads datasets;
4. executes analytics queries;
5. records benchmark metrics.

This workflow allows the complete benchmark process to be executed automatically across multiple configurations.

## 3. Schema Evolution Scenario

The project simulates two dataset versions.

The baseline version (v1) contains the following fields:

- ts
- user_id
- region
- event_type
- value

The second version (v2) introduces one additional nullable field:

- device_type

The main challenge is maintaining compatibility between both versions during reads.

To solve this issue, the reader first loads the schema contract stored in schema.json. All dataset versions are then aligned to a unified schema before validation and querying.

When reading older datasets, missing optional columns are automatically filled with null values. This allows both dataset versions to remain queryable together through the mixed mode.

The mixed reader therefore simulates a simplified schema evolution workflow similar to what can be found in modern cloud data platforms.

## 4. Benchmark Methodology

The benchmark workflow was implemented in bench.py.

Three dataset sizes were used:

| Label | Rows per Version |
|---|---|
| S | 10’000 |
| M | 100’000 |
| L | 1’000’000 |

The assignment originally suggested scaling datasets by storage size. However, due to local hardware constraints, scaling was implemented using row counts instead.

Each benchmark configuration was executed for three reading modes:

- v1
- v2
- mixed

The benchmark records the following metrics:

- upload throughput;
- download throughput;
- listing time;
- object count;
- total stored bytes;
- query runtime;
- rows loaded.

The benchmark was executed locally on:

- Python 3.12
- Windows 11
- Docker Desktop

- MinIO

All datasets were generated using a fixed random seed in order to improve reproducibility.

## 5. Results and Discussion

### 5.1 Mixed-Version Reads

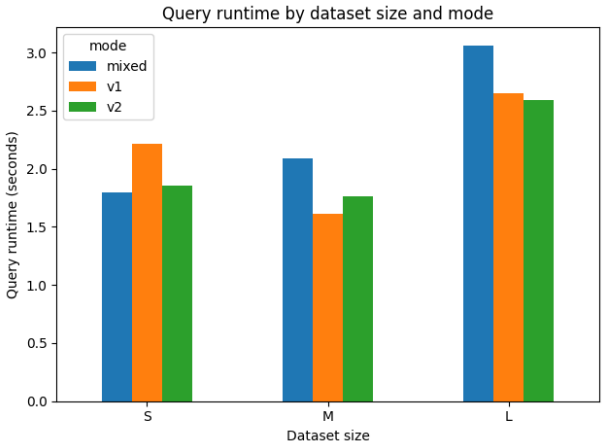
The first objective of the benchmark was verifying that the mixed mode correctly combines both dataset versions.

The benchmark confirms that the mixed mode loads approximately twice as many rows as v1 or v2. This validates that both dataset versions are correctly combined during the read process.

The benchmark therefore confirms that backward-compatible reads work correctly with the current schema evolution scenario.

### 5.2 Query Runtime

The benchmark also evaluates the runtime cost introduced by mixed-version reads.




The results show that the mixed mode generally requires more processing than reading a single dataset version. This behavior is expected because the reader must:

1. load both dataset versions;
2. align schemas;
3. concatenate both tables;
4. execute the analytics query.

The additional cost of the mixed mode becomes more visible for the largest benchmark configuration (L).

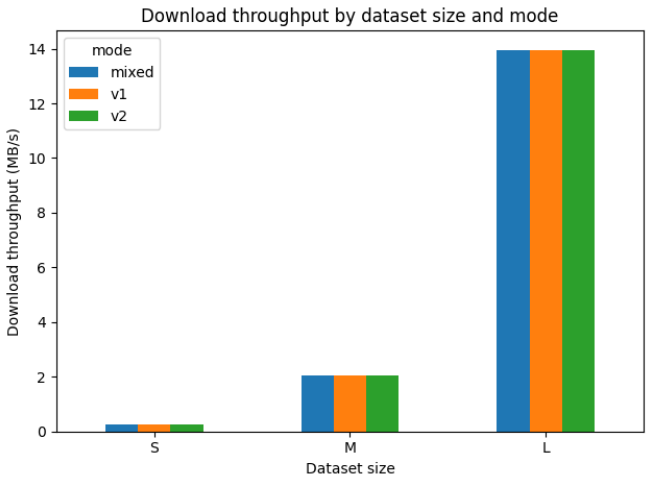
Some runtime variation also remains visible between configurations. Since the benchmark was executed locally, small variations remain expected.

Overall, the results confirm the main trade-off of the implementation:

- mixed-version reads improve compatibility;
- but they introduce additional processing overhead.

### 5.3 Transfer Performance

Upload and download throughput were also measured during the benchmark workflow.



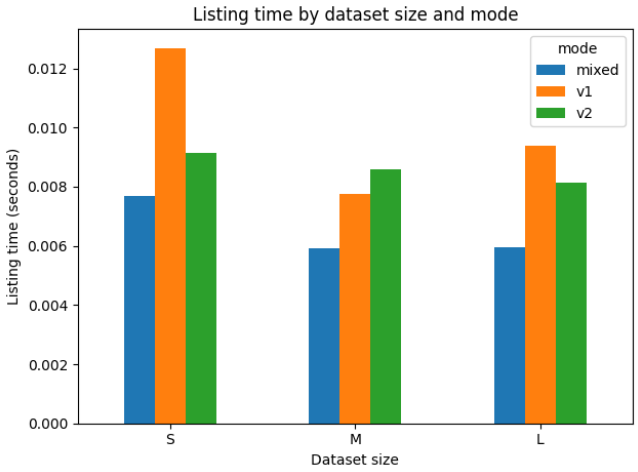

The benchmark shows a clear increase in throughput for larger dataset sizes, while differences between reading modes remain limited.

This suggests that the transfer pipeline behaves consistently across v1, v2, and mixed.

Since the benchmark runs locally with MinIO, the throughput values should be interpreted as relative measurements rather than production cloud performance.

### 5.4 Listing Time and Object Layout

Object listing time remained very small throughout the benchmark.




This behavior is expected because the implementation uses a limited number of Parquet objects per dataset version. In larger storage systems containing many files, this cost would likely become more visible.

The object count also remained coherent across all benchmark configurations:

- v1 reads one object;
- v2 reads one object;
- mixed reads both objects.

This confirms that the reader accesses the expected dataset versions during mixed reads.

## 6. Limitations

Several limitations remain in the current experiment.

First, the benchmark was executed locally using Docker and MinIO. The results therefore do not represent production cloud performance.

Second, only one benchmark run was executed for each configuration. Repeating the benchmark several times and averaging the results would improve measurement reliability.

Third, the schema evolution scenario remains relatively simple because it only introduces one nullable column (device_type).

More complex schema changes would require additional compatibility logic.

Finally, the benchmark uses a small number of Parquet objects per dataset version. Larger storage layouts would likely produce different listing and query behaviors.

## 7. Conclusion

This project demonstrates a complete schema evolution workflow on S3-compatible object storage.

The implementation supports:

- versioned Parquet datasets;
- schema contracts;
- upload and download through the S3 API;
- backward-compatible reads;
- reproducible benchmark execution.

The benchmark confirms that mixed-version reads remain functional across all tested dataset sizes while introducing additional processing overhead compared to single-version reads.

Overall, the workflow executed successfully across all tested configurations and provides a simple but realistic example of schema evolution on object storage systems.